



Интро

- ТЕХНИКА

Проверка связи

Если у вас нет звука

убедитесь, что на устройстве и колонках включён звук
обновите страницу или переподключитесь к вебинару
откройте вебинар в другом браузере
перезагрузите устройство и войдите заново

Пожалуйста, отметьтесь в чате



если меня видно и слышно



если есть проблемы со звуком или видео

- 01 / Десктоп

С компьютера

Используйте **Google Chrome** или **Microsoft Edge**.

При проблемах - обновите страницу **F5**

- 02 / Мобильный

С телефона или планшета

Перейдите с мобильного интернета на **WiFi**.

При проблемах - перезапустите приложение Webinar.

- 03 / Общий совет

Если ничего не помогло

Отключитесь и подключитесь заново.

Пишите в чат - организаторы помогут.

- ЗНАКОМСТВО

Перед началом

- КАКИЕ НОВОСТИ?

Что интересного произошло за неделю?

Напишите в чат - событие, новость или вывод из работы, учёбы или личного опыта.



- ПРЕПОДАВАТЕЛЬ КУРСА

Павлин Николай

Преподаватель кафедры бизнес-информатики МТУСИ
с 2021 года

СТО Nova · стартап в сфере мониторинга здоровья · резидент Сколково

Разрабатываю **LLM-приложения** и RAG-платформу **RAGMaster** (грант ФСИ)

Автор RAG-системы для охраны труда - **chat.kiout.ru**, 1000+ пользователей

Соавтор LLM, попавшей в **топ-20** русскоязычных моделей по MERA (02.2025)

7+ лет в коммерческой разработке: Python, FastAPI, LLM-стек, DevOps

Многократный победитель федеральных хакатонов



Telegram



YouTube



LinkedIn

- ДОГОВОРЁННОСТИ

Правила участия

- 1** Продолжительность вебинара - **80 минут**
- 2** Задавайте **вопросы в чате** - ответу в процессе или в конце
- 3** **Запись** будет доступна на платформе на следующий день
- 4** Обсуждение можно **продолжить в чате** после вебинара
- 5** **Воздерживаемся от обсуждения политических тем.** Здесь мы про разработку, ИИ и продукты - давайте сохраним фокус.

Начинаем!





Паттерны архитектуры ИИ-приложений

Модуль 3 Блок 2

Павлин Н.К.

План на сегодня

Блок	Темы
1. Почему это сложно	Отличия LLM от CRUD, реальные сбои
2. Словарь и слои	Термины + Gateway/Service/LLM/Data
3. Базовые паттерны	Request-Response, Streaming, Queue, Fan-out
4. Отказоустойчивость	Circuit Breaker, Bulkhead, Fallback chain
5. Production-паттерны	Cache-Aside, Batch API, Rate limit, Idempotency, Claim-Check
6. Реальные архитектуры	LLM Gateway, Perplexity, Cursor
7. Итоги и ДЗ	Грабли, дерево выбора паттерна, домашнее задание

Что вы уже знаете — и что добавим

Из модуля 2 (API и клиент):

- Retry с exponential backoff на уровне одного клиента (M2B3)
- Fallback между провайдерами внутри одной функции (M2B3)
- Кеширование ответов в памяти / SQLite (M2B4)
- Простой LLM-ассистент с мультипровайдерностью (M2B5)

Из M3B1 (прошрое занятие):

- Function Calling / Tool Use: JSON Schema, tool_call, tool_choice
- MCP как протокол подключения инструментов

Сегодня добавляем:

- **Архитектурный уровень:** паттерны вне одной функции — между слоями, процессами, сервисами
- **Отказоустойчивость на уровне системы,** а не одного вызова
- **Масштабирование:** stateless, очереди, батчинг, кеш на Redis

Словарь занятия

Термин	English	Что это простыми словами
Request-Response	Request-Response	Классика: запрос → ответ. Клиент ждёт полного ответа
Streaming	Streaming / SSE	Ответ приходит по токenu, пользователь видит «печать»
Queue-based	Queue-based processing	Задача в очередь, <code>task_id</code> на руки, результат позже
Fan-out / Fan-in	Fan-out / Fan-in	Один запрос → N параллельных LLM-вызовов → агрегация
Автомат-предохранитель	Circuit Breaker	После серии ошибок перестаём стучаться к провайдеру
Bulkhead	Bulkhead	Переборки: каждой фиче свой пул ресурсов
Fallback chain	Fallback chain	Цепочка: OpenAI → Anthropic → Ollama
Cache-Aside	Cache-Aside	Сначала кеш → если нет → LLM → записали в кеш
Claim-Check	Claim-Check	В сообщении ID, сами данные — в S3/Redis
Backpressure	Backpressure	Система честно говорит «подожди», а не падает
Rate limiting	Rate limiting	Ограничение входа: token bucket / sliding window
Stateless	Stateless	Сервис без памяти: любой инстанс обработает любой запрос
LLM Gateway	LLM Gateway	Прокси (LiteLLM / Portkey): единый API к N провайдерам

Почему архитектура ИИ-приложений — это сложно

Обычный REST к БД: 1–10 мс, предсказуемо, детерминировано, \$0.0001 за запрос.

Вызов LLM API:

- **Latency:** 1–30 секунд — в 1000× медленнее БД
- **Стоимость:** \$0.01–\$0.50 за запрос, ошибочный retry стоит реальных денег
- **Недетерминизм:** один промпт → разные ответы
- **Rate limits:** OpenAI Tier 1 = 500 RPM, 200K TPM (при больших входах limit'ом становится TPM, не RPM), Anthropic = 1–4K RPM
- **Нестабильность:** 429, 500, 503, overloaded в любой момент
- **Размер ответа:** от 10 токенов до 100K+ — непредсказуемо

Вывод: архитектура ИИ-приложения ≠ архитектура CRUD. Нужны специальные паттерны для медленных, дорогих и нестабильных внешних сервисов.

Реальность: Провайдеры могут быть недоступны

Ноябрь 2024 — OpenAI:

- 8 ноября 2024, 4:06–4:41 PT: ChatGPT и API возвращают 503, затронуты веб, iOS, API
- Несколько инцидентов подряд в ноябре — deployments с новыми моделями роняли API

Апрель 2026 — Anthropic:

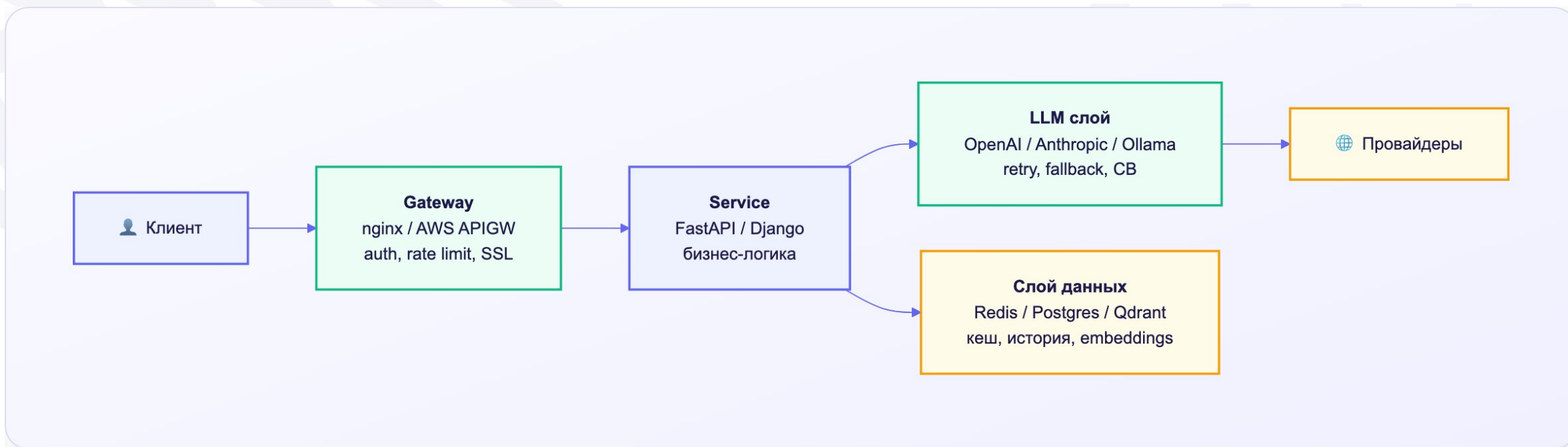
- Частые падения в связи с выросшей нагрузкой

Что это значит для вас:

- OpenAI SLA = 99.9% — это **8.7 часов даунтайма в год**, и они не обязательно равномерные
- Два независимых провайдера: $0.1\% \times 0.1\% = \mathbf{0.0001\%}$ вероятность одновременного падения
- **Вопрос не «если», а «когда»** ваш основной провайдер упадёт

Слои ИИ-приложения

Ключевой принцип: каждый слой заменяется, не затрагивая остальные. Сменили OpenAI на Anthropic — меняется только LLM слой.



Базовые паттерны взаимодействия

Четыре способа, как клиент общается с LLM-сервисом: Request-Response, Streaming, Queue-based, Fan-out / Fan-in.

Что было: разобрали словарь, слои и причины, почему LLM-архитектура сложнее CRUD.

Что добавляем: конкретные паттерны взаимодействия — выбор зависит от длины ответа, UX и нагрузки.

Паттерн 1: Request-Response

Самый простой паттерн: клиент ждёт полного ответа.

```
example.python

1  from fastapi import FastAPI
2  from openai import OpenAI
3
4  app = FastAPI()
5  client = OpenAI(timeout=30.0) # обязательно timeout!
6
7  @app.post("/chat")
8  async def chat(message: str):
9      # Клиент ждёт, пока LLM сгенерирует весь ответ
10     response = client.chat.completions.create(
11         model="gpt-5-mini",
12         messages=[{"role": "user", "content": message}],
13         max_tokens=500,
14     )
15     return {"answer": response.choices[0].message.content}
```

Паттерн 1: Request-Response

Когда подходит: внутренние API, backend-to-backend, короткие ответы (классификация, извлечение данных).

Когда НЕ подходит: чат-боты (пользователь ждёт 5–15 сек без обратной связи), длинные генерации (timeout nginx = 30–60 сек).

Паттерн 2: Streaming — зачем

Первый токен за 200–600 мс, а не через 10 секунд. Пользователь видит «печатающийся» текст — UX как в ChatGPT.

Метрики:

- **TTFT (time-to-first-token):** 200–600 мс — первый кусочек ответа
- **Total time:** 5–15 сек — те же, что у Request-Response
- **Воспринимаемая задержка:** низкая — пользователь уже читает

Формат: Server-Sent Events (SSE) — стандарт HTTP, работает «из коробки» в браузере ([EventSource API](#)).

Где обязателен:

- Чат-боты (Telegram, Web, мобильные)
- Генерация длинных ответов (>500 токенов)
- Интерактивные интерфейсы с ожиданием

Где не нужен: backend-to-backend API, классификация, извлечение данных — пользователю всё равно виден только финальный результат.

Streaming: минимальный SSE в FastAPI

```
example.python

1  from fastapi import FastAPI
2  from fastapi.responses import StreamingResponse
3  from openai import OpenAI
4
5  app = FastAPI()
6  client = OpenAI(timeout=30.0)
7
8  @app.post("/chat/stream")
9  async def chat_stream(message: str):
10     def generate():
11         stream = client.chat.completions.create(
12             model="gpt-5-mini",
13             messages=[{"role": "user", "content": message}],
14             stream=True,
15             stream_options={"include_usage": True}, # токены в финале
16         )
17         for chunk in stream:
18             if chunk.choices and chunk.choices[0].delta.content:
19                 yield f"data: {chunk.choices[0].delta.content}\n\n"
20             yield "data: [DONE]\n\n"
21
22     return StreamingResponse(generate(), media_type="text/event-stream")
```

Streaming: минимальный SSE в FastAPI

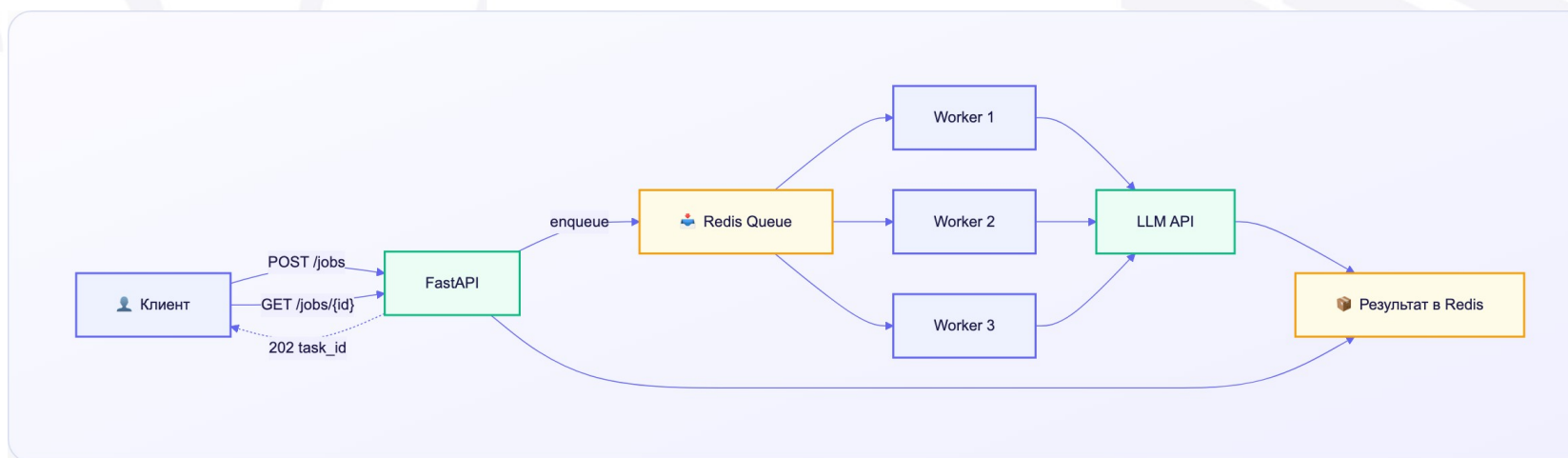
Подводный камень: без `stream_options={"include_usage": True}` токены не придут — `usage` только в финальном chunk'e.

Паттерн 3: Queue-based

Проблема. Ответ LLM может занимать десятки секунд или минуты — отчёты, длинная суммаризация, batch-обработка. HTTP-таймауты (nginx, ALB, браузер) обычно 30–60 секунд, длинный ответ их превышает — клиент получает 504, а worker всё ещё занят.

Решение. Приняли запрос → положили в очередь → вернули `task_id` → обработали фоном → клиент забрал результат через polling / webhook / SSE.

Когда использовать: batch-обработка (500 отзывов), генерация длинных отчётов, превышение rate limits.



Queue-based: код на taskiq

taskiq — нативная async библиотека с типизированными задачами и сменными брокерами (Redis, NATS, Kafka).

example.python

```
1 # tasks.py
2 from taskiq_redis import ListQueueBroker, RedisAsyncResultBackend
3 from openai import AsyncOpenAI
4
5 broker = ListQueueBroker("redis://redis:6379").with_result_backend(
6     RedisAsyncResultBackend("redis://redis:6379"),
7 )
8
9 client = AsyncOpenAI(timeout=60.0)
10
11 @broker.task
12 async def summarize(text: str) -> str:
13     resp = await client.chat.completions.create(
14         model="gpt-5-mini",
15         messages=[{"role": "user", "content": f"Summarize: {text}"}],
16     )
17     return resp.choices[0].message.content
```

example.python

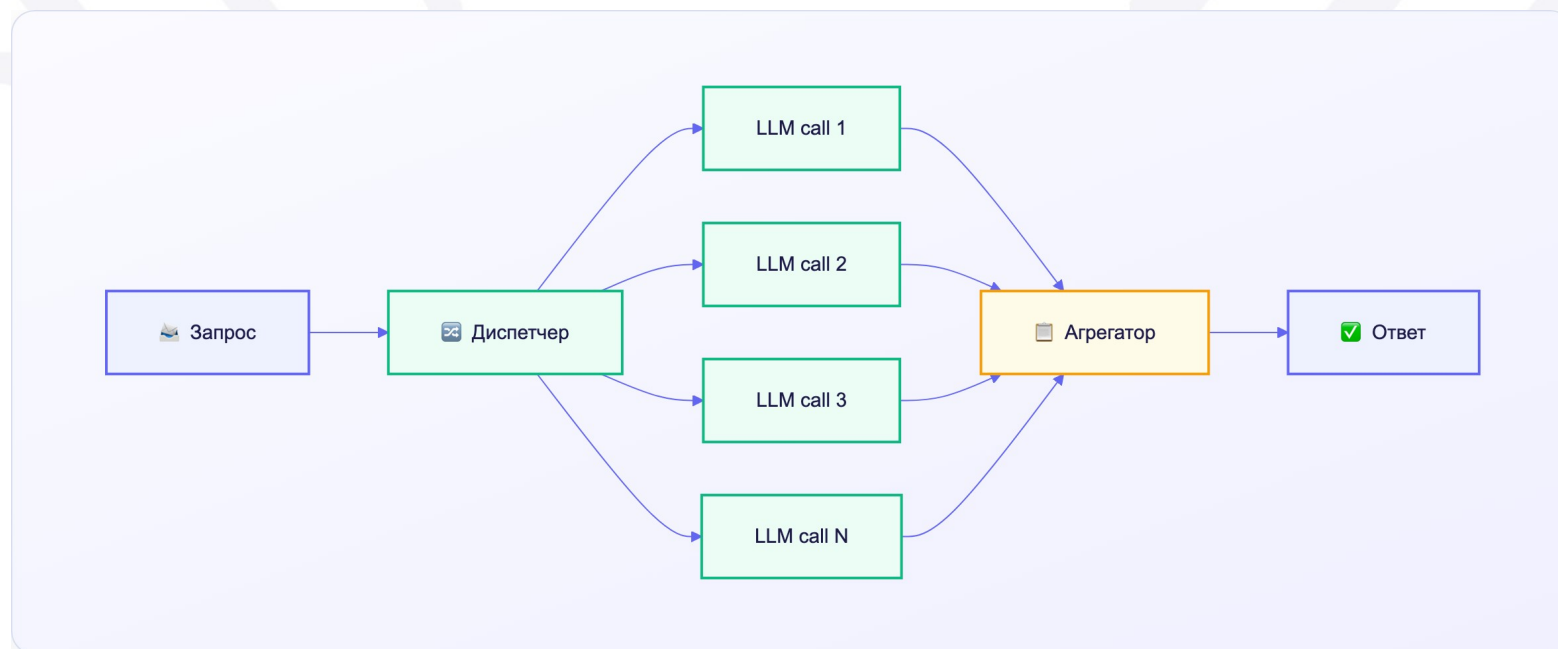
```
1 # api.py
2 from fastapi import FastAPI
3 from tasks import broker, summarize
4
5 app = FastAPI()
6
7 @app.on_event("startup")
8 async def on_startup():
9     await broker.startup()
10
11 @app.post("/jobs")
12 async def enqueue(text: str):
13     task = await summarize.kiq(text) # положили в очередь
14     return {"task_id": task.task_id, "status": "queued"}
```

Паттерн 4: Fan-out / Fan-in

Один запрос пользователя → N параллельных LLM-вызовов → агрегация.

Примеры: анализ 50-страничного PDF (разбить по страницам, параллельно), self-consistency (N прогонов одного промпта, голосование), сравнение GPT vs Claude.

Выигрыш: N запросов по 5 сек = 5 сек параллельно (а не 5N последовательно).



Задача

У вас Telegram-бот, 100 сообщений/минуту. Средний ответ от LLM — 8 секунд.

Вопрос: какую архитектуру выберете? Напишите в чат:

1. Паттерн взаимодействия (RR / Streaming / Queue / Fan-out)
2. Где будет узкое место?
3. Что сделаете, если рост нагрузки в 10×?

Отказоустойчивость: три слоя защиты

Базовые паттерны разобрали — теперь учимся переживать сбои провайдеров.

Что было: Request-Response, Streaming, Queue, Fan-out — как клиент общается с сервисом.

Что добавляем: Circuit Breaker, Bulkhead, Fallback chain — три независимых слоя, которые страхуют от сбоев провайдера, перегрузки и каскадных отказов.

Circuit Breaker

Проблема. OpenAI отвечает 503. Сервис упорно шлёт запросы → каждый висит 30 сек до таймаута → workers заняты → сервис недоступен **для всех** (cascading failure).

Решение. После N ошибок подряд — перестать стучаться. Даём сервису восстановиться.

Параметры: порог ошибок = 5, таймаут открытого = 60 сек, отслеживаемые ошибки = `Timeout`, `APIError`, 5xx.



Circuit Breaker: код на aiobreaker

Рабочий пример через [aiobreaker](#) (async fork [pybreaker](#)):

example.python

```
1 from datetime import timedelta
2 from aiobreaker import CircuitBreaker, CircuitBreakerError
3 from openai import AsyncOpenAI, APIError
4
5 openai_breaker = CircuitBreaker(
6     fail_max=5,                # 5 ошибок подряд
7     timeout_duration=timedelta(seconds=60), # на минуту закрываемся
8     exclude=[ValueError],     # эти не считаем за сбой
9 )
10
11 client = AsyncOpenAI(timeout=30.0)
12
13 @openai_breaker
14 async def call_openai(prompt: str) -> str:
15     resp = await client.chat.completions.create(
16         model="gpt-5-mini",
17         messages=[{"role": "user", "content": prompt}],
18     )
19     return resp.choices[0].message.content
```

example.python

```
1 async def safe_generate(prompt: str) -> str:
2     try:
3         return await call_openai(prompt)
4     except CircuitBreakerError:
5         # СВ открыт — не зовём OpenAI вообще
6         return await call_anthropic_fallback(prompt)
```

Retry с backoff: tenacity

СВ отключает мёртвого провайдера целиком. Но большинство ошибок LLM-API — **переходящие**: rate limit на секунду, разрыв TLS, перегрузка региона. Их лечит обычный retry с экспоненциальным backoff. Удобно — через **tenacity**.

```
example.python

1  from tenacity import (
2      retry, stop_after_attempt, wait_exponential,
3      retry_if_exception_type, before_sleep_log,
4  )
5  from openai import AsyncOpenAI, APIError, RateLimitError
6  import logging
7
8  log = logging.getLogger(__name__)
9  client = AsyncOpenAI(timeout=30.0)
10
11  @retry(
12      stop=stop_after_attempt(3),
13      wait=wait_exponential(multiplier=1, min=2, max=10),
14      retry=retry_if_exception_type((RateLimitError, APIError)),
15      before_sleep=before_sleep_log(log, logging.WARNING),
16      reraise=True,
17  )
18  async def call_openai(prompt: str) -> str:
19      resp = await client.chat.completions.create(
20          model="gpt-5-mini",
21          messages=[{"role": "user", "content": prompt}],
22      )
23      return resp.choices[0].message.content
```

От Circuit Breaker к Bulkhead

Что было: СВ защищает от **внешнего сбоя** — провайдер упал, мы перестаём слать запросы.

Что добавляем: Bulkhead защищает от **внутренней перегрузки** — один тип запросов «съел» все ресурсы процесса и положил соседние endpoint'ы. СВ и Bulkhead не заменяют друг друга, они работают на разных границах.

Инструмент изоляции — `asyncio.Semaphore(N)`. Базовый примитив, на нём построены оба паттерна Bulkhead'a: пул соединений и пул «одновременных вызовов».

- Семафор — счётчик «свободных слотов» с потолком N. `acquire()` забирает один слот, `release()` возвращает. Когда слотов нет — корутина встаёт в очередь и ждёт `release()`.
- В отличие от **Lock** (1 владелец), Semaphore допускает до N одновременных удерживаний — это и есть «пул».
- Идиома production-кода: `async with sem:` — гарантированно отпускает слот даже при исключении.

Bulkhead: изоляция ресурсов

Проблема. У сервиса 3 endpoint'a: `/chat`, `/summarize`, `/classify`. Все дерут один пул подключений. Всплеск суммаризации → все слоты заняты → чат лёг.

Решение. Каждому endpoint — свой пул (семафор, пул соединений).

```
example.python
1 import asyncio
2 from fastapi import FastAPI, HTTPException
3
4 app = FastAPI()
5
6 # Отдельные семафоры для каждого endpoint
7 chat_sem = asyncio.Semaphore(20)      # до 20 параллельных
8 summarize_sem = asyncio.Semaphore(5)  # дорогая операция
9 classify_sem = asyncio.Semaphore(50)   # дешёвая, массовая
10
11 async def acquire_or_503(sem: asyncio.Semaphore, timeout: float = 5.0):
12     try:
13         await asyncio.wait_for(sem.acquire(), timeout=timeout)
14     except asyncio.TimeoutError:
15         raise HTTPException(503, "Service overloaded, try later")
16
17 @app.post("/chat")
18 async def chat(msg: str):
19     await acquire_or_503(chat_sem)
20     try:
21         return await call_llm(msg)
22     finally:
23         chat_sem.release()
```

Fallback chain: цепочка провайдеров

Принцип. Основной упал — идём в резервный. Резервный тоже — в запасной. Ничего не работает — возвращаем честное сообщение.

```
example.python

1 from openai import AsyncOpenAI, APIError
2 from anthropic import AsyncAnthropic
3 import httpx, logging
4
5 log = logging.getLogger(__name__)
6
7 PROVIDERS = [
8     {"name": "openai", "model": "gpt-5-mini"},
9     {"name": "anthropic", "model": "claude-sonnet-4-6"},
10    {"name": "ollama", "model": "qwen3:32b"},
11 ]
12
13 async def generate_with_fallback(prompt: str) -> str:
14     for p in PROVIDERS:
15         try:
16             return await _call(p, prompt)
17         except (APIError, httpx.HTTPError, asyncio.TimeoutError) as e:
18             log.warning(f"{p['name']} failed: {e}")
19             continue
20     return "Сервис временно недоступен. Попробуйте через минуту."
```

Fallback chain: цепочка провайдеров

Выбор порядка:

- По качеству: Anthropic → OpenAI → Ollama
- По стоимости: DeepSeek → GPT-4.1-mini → Claude
- По длине контекста: Claude (200K) → Gemini (1M) → GPT

Обсуждение

В ноябре 2024 OpenAI лежал 35 минут. Ваш сервис полностью зависел от OpenAI.

Вопрос: напишите в чат **одно** архитектурное изменение, которое спасло бы вас.

Варианты (для себя):

- **Fallback chain** — переключение на Anthropic/Ollama
- **Кеш на Redis** — хит-рейт 40% = сервис работает для 40% запросов даже при полном падении
- **Graceful degradation** — не падать, а показывать «базовую версию» без LLM
- **Queue** — принимаем запросы, обрабатываем когда API вернётся

Отказоустойчивость: три паттерна вместе

Circuit Breaker + Bulkhead + Fallback chain — три независимых слоя защиты. Каждый решает свою задачу, вместе они страхуют друг друга.

Правило: применяются **слоями в глубину**. Один без другого — уязвимость.

Паттерн	От чего защищает	Что делает
Circuit Breaker	Мёртвый поставщик	Перестать слать запросы после N ошибок
Bulkhead	Перегрузка одним endpoint'ом	Изолировать пулы ресурсов
Fallback chain	Недоступность поставщика	Переключиться на резервного
Rate Limit (вход)	Злоупотребление одним пользователем	Ограничить N req/min
Timeout	Зависший запрос	Прервать через 30 сек

Production-паттерны: деньги, дубли, большие данные

Сервис уже не падает — теперь оптимизируем стоимость, защищаемся от ретраев и больших нагрузок.

Что было: отказоустойчивость — Circuit Breaker, Bulkhead, Fallback chain.

Что добавляем: Cache-Aside, Semantic cache, Batch API, Rate Limiting, Idempotency keys, Claim-Check — паттерны зрелого LLM-сервиса.

Cache-Aside: экономим деньги и latency

Факт: 20–40% LLM-запросов в типичном приложении — повторяющиеся или похожие.

Cache-Aside (классика): сначала смотрим в кеш → если нет → идём в LLM → кладём ответ в кеш.

```
example.python

1 import hashlib, json
2 import redis.asyncio as redis
3
4 r = redis.Redis(host="redis", decode_responses=True)
5
6 def cache_key(model: str, messages: list, temperature: float) -> str:
7     payload = json.dumps({"m": model, "msgs": messages, "t": temperature}, sort_keys=True)
8     return "llm:" + hashlib.sha256(payload.encode()).hexdigest()
9
10 async def generate_cached(model: str, messages: list, ttl: int = 3600) -> str:
11     key = cache_key(model, messages, 0.0)
12     if cached := await r.get(key):
13         return cached
14     answer = await call_llm(model, messages)
15     await r.setex(key, ttl, answer)
16     return answer
```

Cache-Aside: экономим деньги и latency

Экономика: 10К запросов/день × \$0.002 = \$20/день. Cache-hit 40% → \$12/день. Минус \$240 в месяц.

Semantic cache: хит на «похожих» запросах

Проблема exact match: «Какая погода в Москве?» и «Погода Москва сегодня?» — для хеша это **разные** ключи. Кеш не срабатывает, хотя смысл один.

Semantic cache:

1. Считаем **embedding** запроса (OpenAI `text-embedding-3-small`, \$0.02/1M)
2. Ищем ближайший в векторной БД (Qdrant / pgvector / Redis Vector)
3. Если `cosine similarity > 0.95` — возвращаем cached ответ
4. Иначе — идём в LLM, сохраняем вместе с embedding

Когда окупается: FAQ-боты, техподдержка, классификаторы — много похожих формулировок одной задачи.

Инструменты: GPTCache (Python library), Redis Vector Search, Qdrant, pgvector.

Это тема M5 (RAG) — там разберёмся подробно с embeddings и векторным поиском. Здесь — как архитектурный паттерн.

Batch API: скидка 50% за отложенную обработку

OpenAI / Anthropic Batch API: вы загружаете N запросов как файл, получаете результат **до 24 часов**, платите **50%** от обычной цены.

```
example.python

1  from openai import OpenAI
2  import json
3
4  client = OpenAI()
5
6  # 1. Готовим JSONL с запросами (до 50 000 строк в файле)
7  requests = [
8      {
9          "custom_id": f"req-{i}",
10         "method": "POST",
11         "url": "/v1/chat/completions",
12         "body": {
13             "model": "gpt-5-mini",
14             "messages": [{"role": "user", "content": f"Classify: {text}"}],
15         },
16     }
17     for i, text in enumerate(texts)
18 ]
19 with open("batch.jsonl", "w") as f:
20     for r in requests:
21         f.write(json.dumps(r) + "\n")
```

Batch API: скидка 50% за отложенную обработку

example.python

```
1 # 2. Загружаем и запускаем batch
2 batch_file = client.files.create(file=open("batch.jsonl", "rb"), purpose="batch")
3 batch = client.batches.create(
4     input_file_id=batch_file.id,
5     endpoint="/v1/chat/completions",
6     completion_window="24h",
7 )
8 print(batch.id) # сохраняем, будем забирать результат позже
```

Batch API: скидка 50% за отложенную обработку

Когда использовать: классификация 10K отзывов, генерация описаний товаров, подсчёт метрик — всё, что **не нужно прямо сейчас**.

Rate Limiting на входе

Провайдер лимитирует вас (500 RPM). Вы лимитируете своих пользователей — иначе один пользователь съест весь ЛИМИТ.

Token Bucket в Redis:

```
example.python

1  import time
2  import redis.asyncio as redis
3  from fastapi import HTTPException
4
5  r = redis.Redis(host="redis", decode_responses=True)
6
7  async def rate_limit(user_id: str, max_per_minute: int = 20) -> None:
8      key = f"rl:{user_id}:{int(time.time() // 60)}"
9      current = await r.incr(key)
10     if current == 1:
11         await r.expire(key, 60)
12     if current > max_per_minute:
13         raise HTTPException(429, "Too many requests")
14
15 # Использование в endpoint
16 @app.post("/chat")
17 async def chat(msg: str, user_id: str):
18     await rate_limit(user_id, max_per_minute=20)
19     return await generate_with_fallback(msg)
```

Graceful degradation: падать красиво

Идея. Когда ничего не работает — не возвращать 500, а **деградировать** до более простого ответа.

Уровни деградации LLM-сервиса:

1. **Основной LLM** работает → полный ответ с contextual reasoning
2. **Основной упал**, работает **резервный LLM** → ответ, возможно менее качественный
3. **Все LLM упали**, работает **кеш** → похожий ответ из истории
4. **Кеш пуст**, работает **template** → шаблонный ответ без LLM («Не могу ответить прямо сейчас. Попробуйте сформулировать иначе или напишите в поддержку»)
5. **Ничего не работает** → честный 503 с **Retry-After: 30**

Пример. FAQ-бот: вместо «Сервис недоступен» — показать ссылку на FAQ, форму «написать в поддержку», топ-10 популярных вопросов.

Stateless + внешнее состояние

Принцип. Сам FastAPI ничего в памяти не хранит (кроме конфига). Любое состояние — во внешнем хранилище.

Что где храним:

Зачем. Рестарт без потерь. `docker-compose up --scale app=5` работает из коробки. Rolling deploy без downtime.

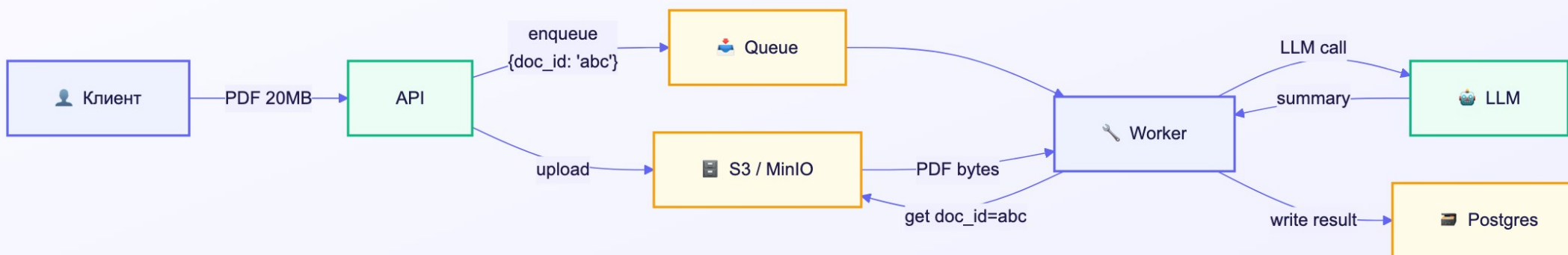
Данные	Где	TTL
Кеш LLM-ответов	Redis	1–24 часа
Сессии пользователей	Redis	30 минут
Rate limit counters	Redis	1 минута
Состояние Circuit Breaker	Redis	30–60 секунд
История диалогов	Postgres	навсегда
Метрики use / cost	Postgres	90–180 дней
Embeddings (RAG)	Qdrant / pgvector	навсегда

Claim-Check: ID вместо тела

Проблема. У вас распределённая система: API → Queue → Worker → LLM. В задаче — PDF на 20 МБ. Имеет смысл хранить файл отдельно, не в БД.

Решение Claim-Check: большие данные кладём в S3/MinIO, в сообщении передаём **только ID** (claim check = номерок в гардеробе).

Для LLM актуально: большие документы в RAG, multimodal (картинки, аудио), длинные истории чатов.



Как это собрано у других

Паттерны разобрали по отдельности — посмотрим, как они складываются в реальных продуктах.

Что было: Cache-Aside, Batch, Rate Limit, Idempotency, Claim-Check, graceful degradation.

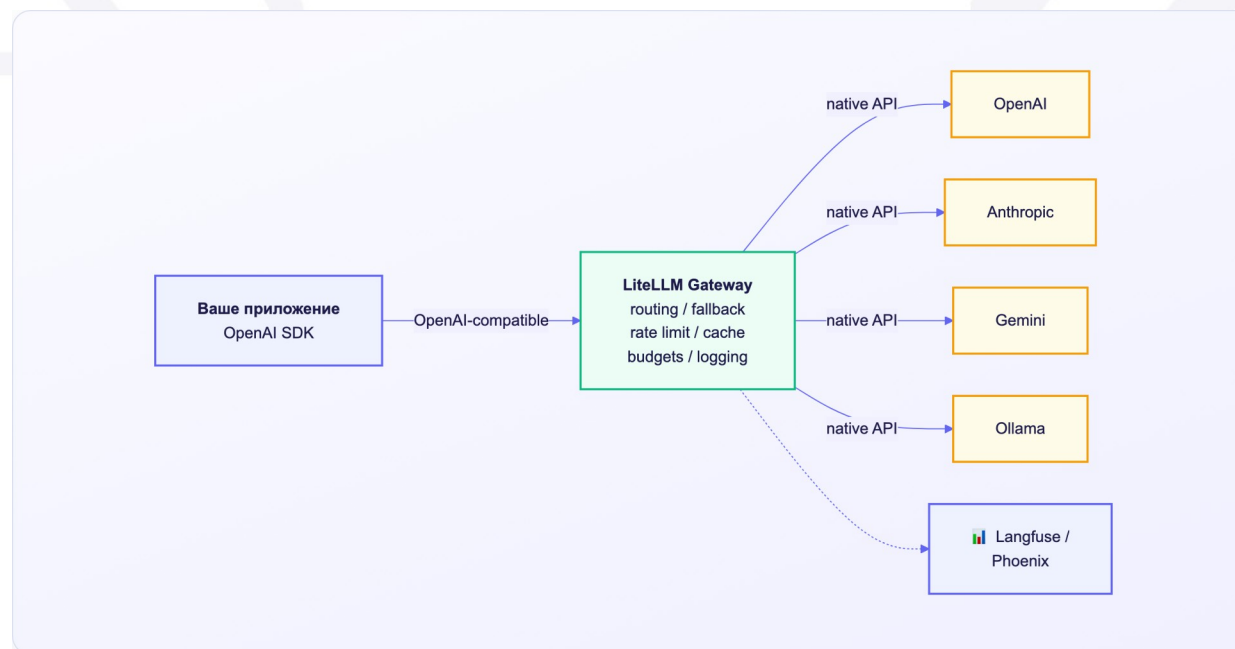
Что добавляем: архитектуры LiteLLM, Perplexity и Cursor — как зрелые LLM-продукты комбинируют сегодняшние паттерны.

Реальная архитектура: LLM Gateway (LiteLLM)

LLM Gateway — прокси между вашим кодом и N LLM-провайдерами. Единый API, смена провайдера — одна строка.

Что встроено: fallback chains, routing по cost/latency, кеш, rate limit per team, cost tracking, observability.

Альтернативы: Portkey (managed), OpenRouter (SaaS, pay-per-token), Requesty, Bifrost.



LiteLLM: config вместо кода

Вместо того чтобы писать свой CB + fallback + cache, описываете это в YAML:

```
example.yaml
1 # litellm-config.yaml
2 model_list:
3   - model_name: smart      # ваш "logical" name
4     litellm_params:
5       model: openai/gpt-5-mini
6       api_key: os.environ/OPENAI_API_KEY
7     model_info:
8       rpm: 500
9   - model_name: smart      # тот же logical — автоматический load balance
10    litellm_params:
11      model: anthropic/claude-sonnet-4-6
12      api_key: os.environ/ANTHROPIC_API_KEY
13    model_info:
14      rpm: 1000
15
16 router_settings:
17   routing_strategy: usage-based-routing-v2
18   fallbacks: [{"smart": ["ollama/qwen3:32b"]}]}
19   num_retries: 2
20   timeout: 30
```

```
example.yaml
1 litellm_settings:
2   cache: true
3   cache_params: {type: redis, host: "redis", port: 6379}
```

LiteLLM: config вместо кода

Ваш код: `client = OpenAI(base_url="http://litellm:4000").`

Реальная архитектура: Perplexity и Cursor

Perplexity (AI-поиск, 400М+ запросов/месяц):

- Собственный inference-стек на NVIDIA H100
- **KV-cache aware routing** — запросы с общим префиксом идут в тот же pod (cache hit 87%)
- Rust для hot-path (serving logic, batch scheduling)
- Мультимодальный поток: query → search → rerank → LLM synthesis (Fan-out/Fan-in)

Cursor (AI-редактор кода):

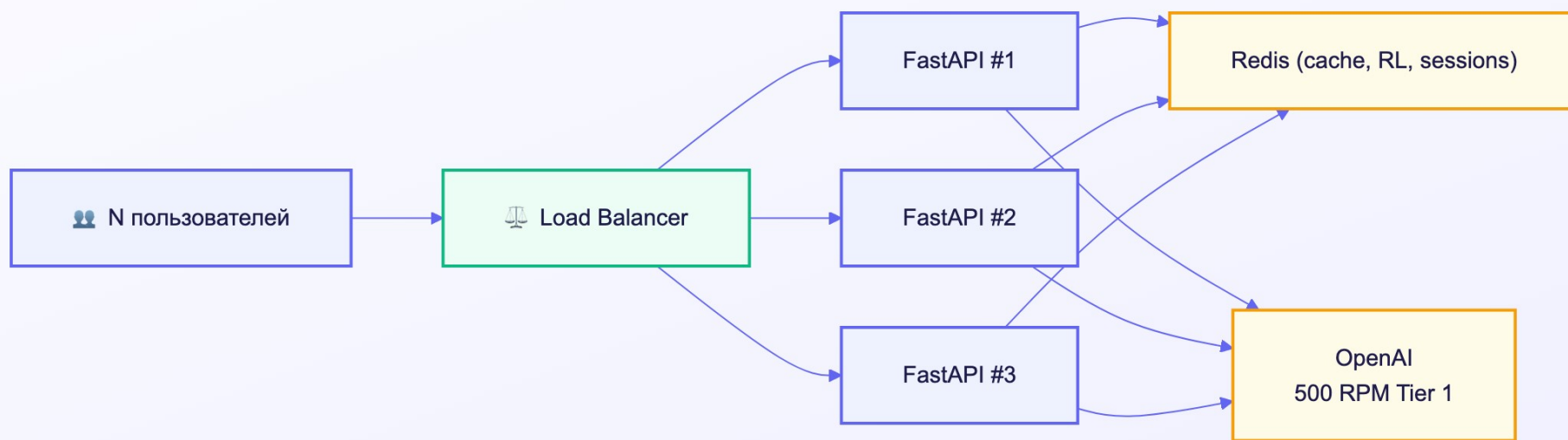
- **Все запросы идут через backend Cursor**, не напрямую к OpenAI/Anthropic
- Backend: препроцессинг (индексация репозитория), роутинг по задаче (completion / chat / agent)
- Свой tensorzero-подобный шлюз для моделей (multi-provider, fallback, A/B)
- Подмешивание контекста из IDE (открытые файлы, LSP) — **это их секрет качества**

Тейкавей: реальные продукты — это не «OpenAI SDK в обработчике». Это слои: препроцессинг → роутинг → LLM → постпроцессинг → кеш → observability.

Масштабирование: узкое место — не вы, а LLM

Парадокс масштабирования LLM: ваши инстансы легко масштабировать горизонтально. Все они упрутся в **один rate limit** провайдера.

Стратегии обхода: повышение tier, несколько API-ключей (если ToS разрешает), мультипровайдерность, кеш (лучшее масштабирование = не делать запрос).



Частые грабли — и как не наступить

1. **Нет таймаута на LLM-вызов** → один висячий запрос держит worker'.
2. **Retry без backoff** → DDoS провайдера собой же → ban. → exponential: 1, 2, 4, 8 сек (M2B3).
3. **Один провайдер без fallback** → OpenAI упал = вы упали. → fallback chain + CB.
4. **Состояние в памяти FastAPI** → `sessions = {}` → рестарт = всё потеряли; 2 worker'a = у каждого свой. → Redis / Postgres.
5. **Нет Semaphore на параллельные вызовы** → 1000 запросов → 429 → всё легло. → `asyncio.Semaphore(50)`.
6. **Кеш на temperature > 0** → возвращаем старый ответ там, где должно быть разнообразие. → кешировать только `temperature=0`.
7. **Синхронный клиент в async handler** → блокирует event loop..
8. **Один CB на все провайдеры** → падение Anthropic открывает CB для OpenAI. → **один CB на один провайдер**.

Дерево выбора паттерна: тип ответа и масштаб

Шаг 1. Тип ответа:

- Короткий (< 200 токенов): Request-Response
- Длинный (> 500 токенов), interactive: Streaming
- Очень длинный / batch: Queue-based

Шаг 2. Количество LLM-вызовов:

- Один: RR или Streaming
- Несколько параллельных: Fan-out / Fan-in
- Много (100+): Queue + Fan-out

Дерево выбора паттерна: надёжность и бюджет

Шаг 3. Требования к надёжности:

- Прототип: RR + один провайдер
- MVP: + Streaming + Fallback chain
- Production: + Circuit Breaker + Bulkhead + Cache-Aside
- Enterprise: + Idempotency + Claim-Check + LLM Gateway

Шаг 4. Бюджет:

- Стартап: кеш + дешёвые модели + Ollama fallback
- Scale-up: Batch API для non-realtime + prompt caching
- Enterprise: мультипровайдер + observability + dedicated tier

ИТОГИ

Что вы теперь умеете:

- Выбирать паттерн: Request-Response / Streaming / Queue / Fan-out
- Проектировать слои: Gateway → Service → LLM → Data
- Применять отказоустойчивость: Circuit Breaker, Bulkhead, Fallback chain
- Использовать production-паттерны: Cache-Aside, Batch API, Rate Limit, Idempotency, Claim-Check
- Читать архитектуру реальных продуктов (LiteLLM, Perplexity, Cursor)

Дальше в модуле — чистая практика: реализуем эти паттерны в коде.

Дополнительные материалы

Боевые архитектурные кейсы:

- [OpenAI — Scaling PostgreSQL to power 800M ChatGPT users](#) — 800–900M WAU, single-primary + ~50 replicas, без шардирования, ~1M qps, p99 в низких двузначных миллисекундах
- [NATS Architecture and Design](#) — публичный репозиторий ADR реальной распределённой системы, образец, как писать architecture decision records

Книга для углубления:

- [Machine Learning System Design \(Babushkin & Kravchenko, Manning, 2025\)](#) — системный учебник по дизайну ML- и LLM-приложений

Референсы

Архитектура и паттерны:

- [Microsoft Cloud Design Patterns](#) — Circuit Breaker, Bulkhead, Queue-Based Load Leveling, Claim-Check, Cache-Aside
- [Circuit Breaker Pattern](#)
- [Bulkhead Pattern](#)

Инструменты:

- [LiteLLM](#) — open-source LLM Gateway
- [Portkey](#) — managed AI Gateway
- [aiobreaker](#) — async Circuit Breaker для Python
- [ARQ](#) — async job queue на Redis
- [OpenAI Batch API](#) — 50% скидка